ETIS

Engineering Studio Executive Overview

Purpose, educational model, stakeholder value, technical posture, governance, and adoption guidance

A leadership and institutional overview of ETIS Engineering Studio and its place in the broader ETIS Engineering Education Suite.

Executive purpose
This document is for academic leaders, instructors, institutional adopters, technical owners, and governance stakeholders who need to understand why Studio exists, how it is bounded, what it requires to operate responsibly, and where to find detailed implementation guidance.

Document control	Value
Document	01 - ETIS Engineering Studio Executive Overview
Status	Production publication
Reference implementation	COMP 330/474, Fall 2026
Companion index	00 - Master Manual & Documentation Index
Primary audiences	Academic leadership, course owners, instructors, adopters, technical administrators, operations owners

Executive Overview

ETIS Engineering Studio is a browser-based engineering apprenticeship environment that helps students practice evidence-based engineering judgment against the real state of their team repository. It is designed for an era in which code can be produced quickly—including with generative AI—but engineering responsibility cannot be automated away. Students remain responsible engineers. AI reviewers are bounded advisers that challenge assumptions, surface evidence, expose uncertainty, and coach reasoning; they do not make hidden team decisions or assign grades.

Studio is intentionally only one part of a larger ETIS educational architecture. The ETIS Framework provides engineering discipline and lifecycle concepts. The Engineering Platform provides guidance and reference material. Studio provides repeated developmental review. ETIS Preflight answers a narrower readiness question near a phase gate. The Engineering Review Center supports instructor-facing formal phase-gate review based on a designated Git tag. These boundaries keep coaching, readiness, and formal academic evaluation from silently collapsing into one AI-driven workflow.

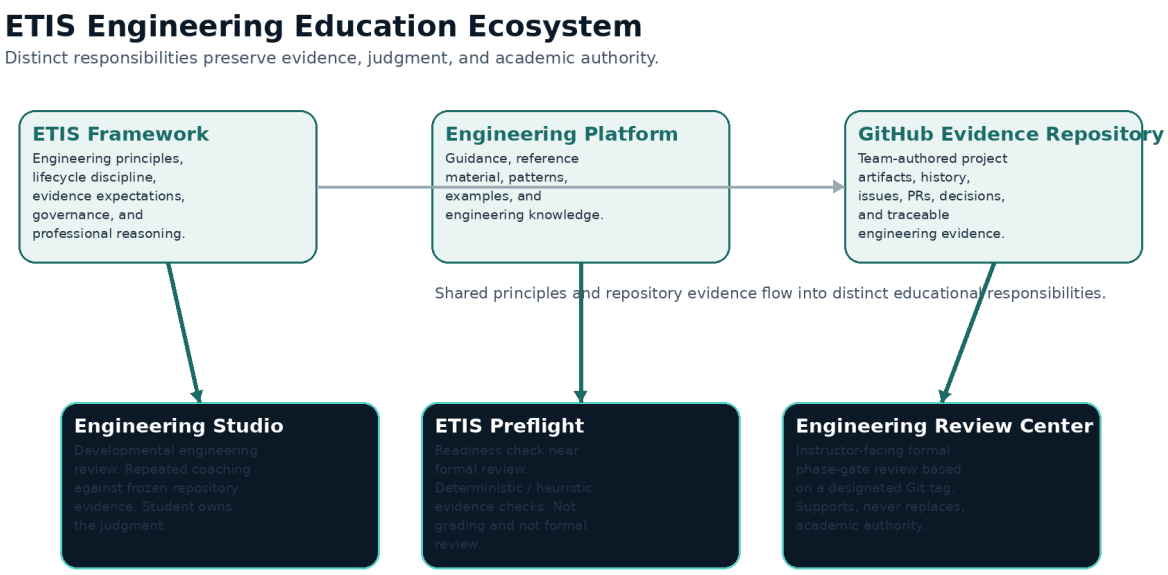


Figure 1. ETIS Engineering Education ecosystem and responsibility boundaries.

The central principle
Students may receive substantial coaching, but the durable engineering record must live in the repository and be defensible without the conversation that helped produce it.

1. Motivation: engineering judgment in an AI-assisted world

The educational problem Studio addresses is not simply whether students can produce software. Modern tools can generate code, tests, documentation, diagrams, and design suggestions at extraordinary speed. The harder question is whether the engineer can explain what should be built, why the evidence is sufficient, what could fail, who owns the consequence, what remains uncertain, and when the decision should change.

- Move students from answer-seeking to evidence-backed engineering reasoning.
- Make uncertainty, tradeoffs, ownership, consequence, and change triggers explicit.

- Teach students to challenge reviewers and AI rather than treating them as authorities.
- Reward durable repository evidence rather than transient conversation quality or raw activity counts.
- Create repeated practice before a formal phase-gate evaluation, so failure can become learning rather than merely a score.

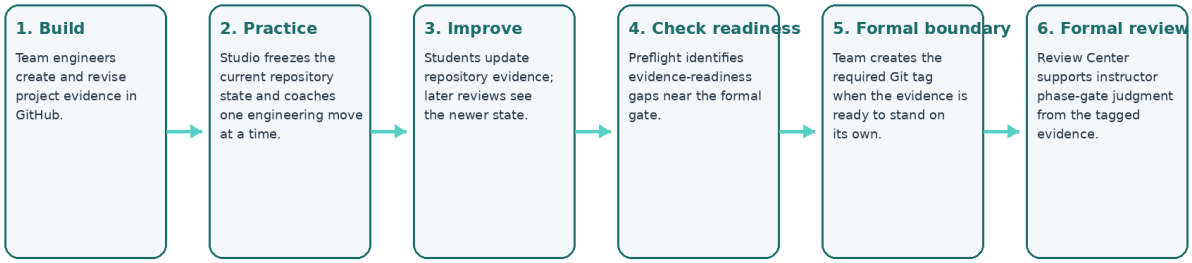
The resulting environment resembles a senior engineering review more than a chatbot. Students can ask questions, disagree, request help, admit they do not know yet, and revise a recommendation as reasoning develops. The pedagogical aim is ownership: the learner should leave the conversation better able to defend the engineering decision independently.

2. Product boundaries and the ETIS suite

Capability	Primary question	Evidence boundary	Academic role
Engineering Platform	What guidance, principles, examples, and references help me engineer this well?	Reference/guidance environment; not the project evidence record.	Supports learning and engineering practice.
Engineering Studio	What should we examine, challenge, or decide while the work is still developing?	Frozen snapshot of the team repository at the start of a review.	Developmental coaching; not grading.
ETIS Preflight	Are we ready to present the evidence for formal phase-gate review?	Deterministic/heuristic checks against current phase expectations.	Readiness only; not formal review and not grading.
Engineering Review Center	What does the formally submitted evidence support at the phase gate?	Designated Git tag establishes the formal evidence boundary.	Supports instructor judgment; does not replace academic authority.

The separation is deliberate. Studio dialogue does not silently become formal evidence. A student can learn through a difficult conversation, improve the repository, use Preflight near readiness, and then create the required formal tag. The Review Center evaluates the tagged repository evidence—not the private developmental conversation—as the formal boundary.

Studio Operating Model: From Student Work to Durable Evidence



Key boundary: Studio conversations help students improve the work; formal review evaluates the tagged repository evidence, not the coaching transcript.

Figure 2. Typical evidence-and-review operating model.

3. Stakeholder experience and value

Stakeholder	What they do	What Studio provides	What Studio does not do
Student engineer	Builds evidence, links team repository, starts developmental reviews, asks/challenges, states recommendations, revisits history.	Structured coaching, evidence visibility, bounded reviewer perspectives, durable history of personal reasoning.	Does not make the team decision or generate a hidden “correct answer.”
Instructor / TA / Reviewer	Monitors teams, inspects shared evidence, views authorized persisted review history, coaches and intervenes.	Class engineering intelligence, attention signals, review visibility, evidence drill-down.	Does not turn activity metrics or AI signals into an automatic grade.
Course Owner	Configures semester, sections, roster, teams, release calendar, staff roles, repository recovery, lifecycle state.	Course operations and historical preservation.	Does not erase history when students move teams or when a term is archived.
Technical Administrator	Configures identity, GitHub integration, Azure, database, Key Vault, deployment environment, monitoring.	Repeatable production installation and bounded administrative controls.	Does not bypass repository/IaC authority through routine portal edits.
Production Operator / Recovery Lead	Runs acceptance, investigates alerts, manages incidents, restores service when needed.	CLI, runbooks, observability, rollback/PITR procedures.	Does not treat cost telemetry or one signal as equivalent to application health.

Shared vs. individual state
Team membership and repository evidence are shared. Developmental coaching conversations and individual learning state remain individual, with visibility limited to the student and authorized teaching staff.

4. Evidence, checkpoints, review history, and formal boundaries

Studio is evidence-bound. When a review begins, the current repository state is frozen as a review snapshot associated with a repository commit. Review findings and discussion are therefore traceable to what the board could actually see at that moment. Later commits do not rewrite the past; a later review sees the newer repository state and creates a newer checkpoint.

- Git history shows how the repository changed over time.
- A Studio snapshot shows the repository state used for a specific developmental review.
- Review History preserves the student’s review sessions and their reasoning state over time.
- Findings distinguish observed facts from review interpretations and can be discussed, challenged, resolved, accepted, or deferred according to the product workflow.

- The formal phase-gate boundary is established separately by the required Git tag used by the Review Center.

This model prevents a common educational failure: allowing a polished conversation to substitute for durable engineering evidence. The team must still improve the repository. The review helps the learner decide what evidence is missing, weak, conflicting, or consequential; it does not silently manufacture that evidence.

5. Semester governance and academic operating model

The reference implementation treats semester state as an authorization and records boundary. A term in setup supports administrative preparation. An active term supports normal student/team/review operations. Archiving ends current operational authority while preserving historical evidence, reviews, findings, team membership history, and usage records.

1. Create the term and one or more sections.
2. Import the roster; create teams; assign or move students while preserving membership history.
3. Configure A1–A6 availability, due, accept-until, and release behavior by section; Sakai remains the course deadline authority in the COMP 330 implementation.
4. Assign the minimum teaching-staff role needed: Course Owner, Instructor, TA, or Reviewer.
5. Activate only after identity, roster, team, repository, schedule, and staff checks are complete.
6. Operate the semester through adds, drops, team moves, repository recovery, schedule adjustments, and evidence/review monitoring.
7. Archive after the term ends so historical records remain available without granting stale operational authority.

Academic authority
Studio can surface evidence and reasoning. The instructor remains the academic decision-maker. No dashboard count, AI response, token total, commit count, or model recommendation is a grade.

6. Technical and production posture

The reference deployment is a production-engineered cloud application, not a classroom-only prototype. Institutional Microsoft Entra identity provides sign-in; roster and role data determine course authorization; GitHub identity is linked separately for engineering provenance; a GitHub App receives bounded read-only repository access; PostgreSQL persists course, evidence, review, and operational state; Azure Container Apps hosts the application; Key Vault protects secrets; Application Insights and Azure Monitor provide observability; GitHub Actions performs controlled deployment from the repository.

Area	Reference posture
Deployment	Branch -> pull request -> CI -> merge -> GitHub Deploy Azure workflow. No routine deployment command in the operator CLI.
Infrastructure authority	Repository Bicep/IaC is authoritative. Runtime overrides are exceptional and can be overwritten by later deployment.
Production scale	Accepted baseline: minimum 1 replica, maximum 5, Single revision mode, external ingress on target port 8000.
Database	Azure Database for PostgreSQL Flexible Server with Alembic-managed schema and point-in-time recovery capability.
Secrets	Azure Key Vault plus protected GitHub production environment; secret values are not printed by routine tooling.

Area	Reference posture
Operations	`./scripts/azure/etis-azure` provides status, health, replicas, revisions, logs, cost, budget, smoke, drift, acceptance, and guarded scale operations.
Recovery	Immutable image rollback, PostgreSQL PITR, configuration/secret recovery, and post-recovery production acceptance.

Operations deliberately separate required health checks from advisory telemetry. For example, Cost Management may throttle or lag without indicating application failure. Production acceptance requires the operational checks that establish service correctness—status, health, replica capacity, external smoke behavior, drift, and budget controls—while cost telemetry remains informational.

7. Security, privacy, and authority boundaries

- Authentication and authorization are separate. Signing in proves identity; course/section/team/staff state determines what that identity may access.
- Student repository access is exact and team-scoped. A repository is nominated, owner authorization is completed where required, and Studio separately verifies exact repository access.
- GitHub App permissions are read-only and limited to the evidence required for engineering review.
- Evidence snapshots, review turns, findings, and lifecycle history are preserved rather than silently rewritten.
- Archived semesters are historical and non-operational; stale staff or student authority does not carry forward as current access.
- Secrets and credentials are handled through protected configuration surfaces and are intentionally omitted from routine reports.
- The product fails closed where authority matters: an unknown, stale, or unverified state should not silently grant access.

8. Institutional adoption and ownership

Adopting Studio is not only a hosting decision. An institution must assign ownership for identity, GitHub organization/App administration, course operations, production monitoring, incident response, recovery, privacy/retention policy, and academic governance. The product is strongest when those responsibilities are explicit before students begin using it.

Decision area	Questions leadership should resolve before go-live
Academic model	Which courses/phases use developmental Studio review? What remains formal instructor evaluation? How will students be taught the distinction?
Identity & access	Which institutional identities are accepted? Who owns roster and staff authorization? What is the exception process?
GitHub governance	Who owns the course organization and GitHub App? Who can approve organization-level repository access?
Data & retention	What student/review/evidence records are retained, for how long, and who may access archived records?
Operations	Who receives alerts, runs acceptance after deployment, owns incidents, and approves emergency runtime changes?

Decision area	Questions leadership should resolve before go-live
Recovery	Who is authorized to perform rollback/PITR and who validates integrity before return to service?
Cost	What budget and alert thresholds are appropriate, and who receives notifications?

9. What successful use looks like

- Students increasingly defend decisions with evidence, consequences, ownership, uncertainty, and change conditions.
- Teams improve durable repository evidence after developmental reviews rather than treating the conversation itself as the deliverable.
- Instructors use attention signals to decide where to look, then exercise human academic judgment from the evidence and context.
- Formal phase-gate review remains anchored to a clear tagged evidence boundary.
- Administrators can move students, recover repository onboarding, and archive terms without destroying history.
- Operators can tell the difference between application failure, configuration drift, advisory cost telemetry, and recoverable external-service degradation.
- Production changes are traceable through repository, CI/CD, deployment, and acceptance rather than ad hoc portal mutation.

A useful test
If Studio disappeared temporarily, the team should still possess the durable engineering evidence in GitHub, and the instructor should still retain academic authority. Studio improves the engineering process; it is not the sole source of truth for the project or the grade.

10. Documentation library and where to go next

No.	Document	Use it when...
00	Master Manual & Documentation Index	You need the overarching product manual, complete library map, terminology, audience routing, and first-adoption checklist.
01	Executive Overview	You need a concise leadership/adoption view of purpose, boundaries, value, governance, and operating posture.
02	Installation & Administration Guide	You are installing or configuring identity, GitHub, Azure, database, secrets, deployment, DNS/TLS, or technical administration.
03	Architecture & Detailed Design	You need trust boundaries, data/evidence model, orchestration, lifecycle, security, failure behavior, topology, and implementation invariants.
04	Instructor & Course Owner Handbook	You are teaching with Studio, monitoring teams, inspecting evidence/reviews, resolving student issues, or using instructor surfaces.

No.	Document	Use it when...
05	Student User Guide	You are a student learning the complete Studio workflow, evidence model, review types, checkpointing, history, findings, and troubleshooting.
06	Team GitHub & Repository Setup Quickstart	Your team is linking GitHub and authorizing/verifying the repository for the first time.
07	Student Cheat Sheet / Quick Reference	You need a short operational reminder while using Studio.
08	Azure Operations CLI User Guide	You are using the `etis-azure` production CLI and need command, output, option, and troubleshooting detail.
09	Production Operations Runbook	Production is unhealthy, an alert fired, or you need routine/pre-class operational checks and incident triage.
10	Semester Administration & Course Operations Guide	You are creating a term, importing roster, managing teams/staff/calendar, handling changes, or archiving.
11	Backup, Recovery & Disaster Recovery Runbook	You need rollback, PITR, secret/configuration recovery, disaster recovery, validation, or return-to-service procedures.

Appendix A - Executive terminology

Term	Meaning
Developmental review	A repeatable Studio review intended to improve engineering judgment and repository evidence while work is still developing.
Formal phase-gate review	Instructor-facing evaluation of the designated tagged repository evidence through the Engineering Review Center.
Frozen evidence snapshot	The repository state captured for a particular Studio review so findings and discussion remain traceable to what the board saw.
FACT	An evidence observation grounded in repository state or other permitted evidence.
REVIEW	An engineering interpretation or concern that can be discussed, challenged, resolved, accepted, or deferred.
CourseTerm status	Authoritative semester lifecycle state: setup, active, or archived.
Production acceptance	Required operational checks establishing that the deployed system matches the accepted production baseline after a deployment or recovery.

Appendix B - Executive go-live questions

- Do students understand that Studio coaches judgment and does not grade them?

- Do instructors understand the distinction between developmental Studio reviews and formal Review Center evaluation?
- Are institutional identity, roster authorization, staff roles, and GitHub ownership responsibilities explicit?
- Are repository authorization and evidence boundaries understood by both students and course staff?
- Is the semester configured, activated, and scheduled intentionally?
- Is production deployment controlled through repository/CI/CD rather than ad hoc mutation?
- Are alerts, budget controls, operations ownership, and recovery authority assigned?
- Has the production acceptance run passed after the final deployment?
- Is there a documented fallback if Studio is temporarily unavailable so students can continue engineering work in GitHub?
- Are retention, privacy, archive, and post-semester responsibilities understood?

ETIS Engineering Studio

Engineering judgment remains a human responsibility.